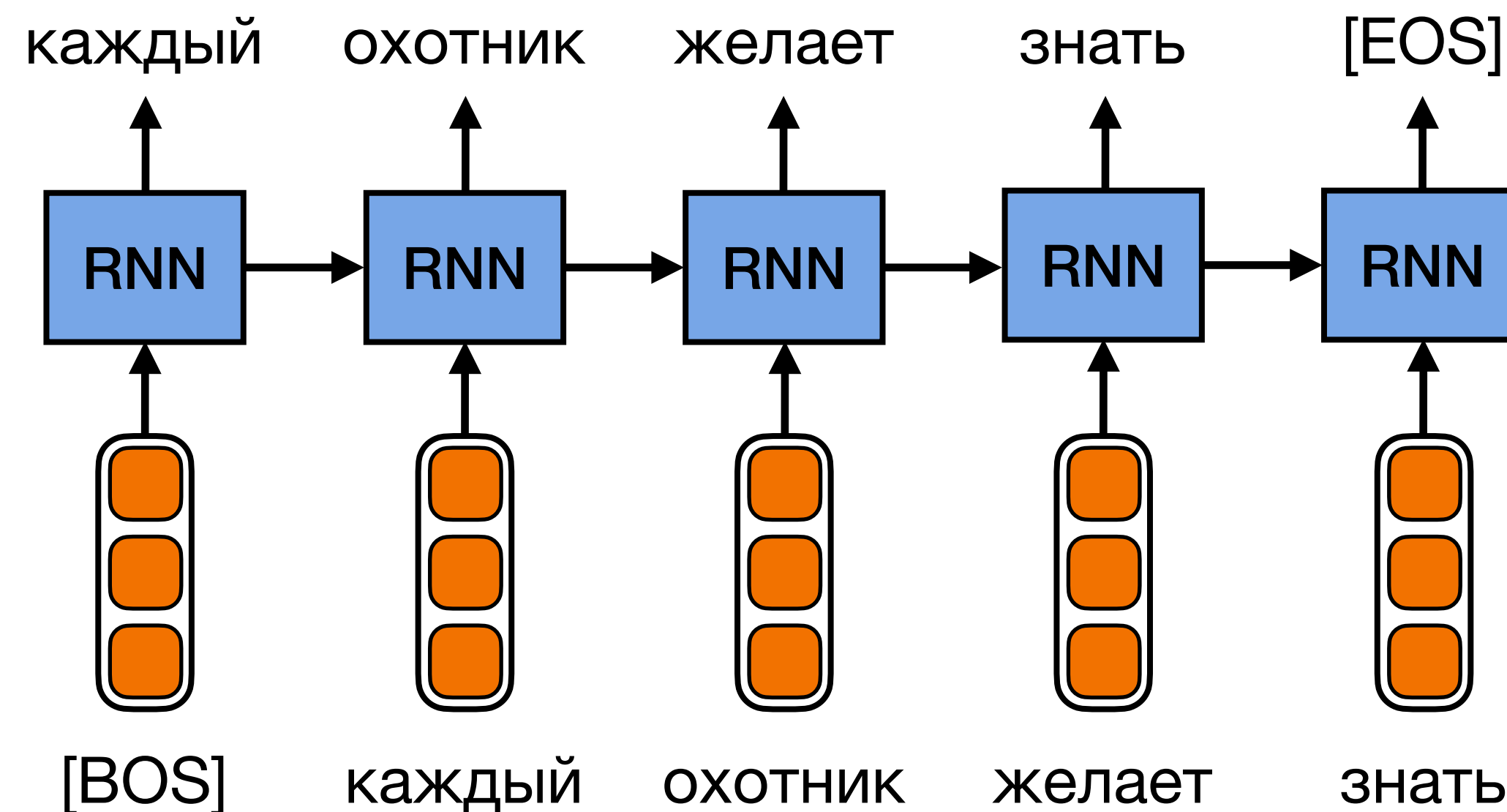


Генерация текста, Seq2seq

Recurrent Neural Networks (RNN)

- Каждый блок предсказывает вероятность следующего токена.



Метрики качества генерации

- Статистические метрики
- Перплексия

Статистические метрики

Необходимо измерить, насколько текст похож на человеческий

Идея:

- Посчитаем различные статистики сгенерированных текстов.
- Сравним их с текстами из корпуса.

Статистические метрики

Модель **повторяет** одно и то же или генерирует **разные** тексты?

Обучающий корпус

Вот дом,
Который построил Джек.
А это пшеница,
Которая в тёмном чулане хранится
В доме,
Который построил Джек.

Генерация

Дом, который построил Джек.
Дом, который построил Джек.
Дом, который построил Джек.

Статистические метрики

Модель **повторяет** одно и то же или генерирует **разные** тексты?

Обучающий корпус

Вот дом,
Который построил Джек.
А это пшеница,
Которая в тёмном чулане хранится
В доме,
Который построил Джек.

Генерация

Дом, который построил Джек.
Дом, который построил Джек.
Дом, который построил Джек.

Отношение числа уникальных n-грамм к
числу всех n-грамм.

$$\frac{|\text{unique 2-gram}|}{|\text{all 2-gram}|} = \frac{3}{9}$$

Чем выше, тем больше разнообразие

Статистические метрики

Модель **повторяет** одно и то же или генерирует **разные** тексты?

Обучающий корпус

Вот дом,
Который построил Джек.
А это пшеница,
Которая в тёмном чулане хранится
В доме,
Который построил Джек.

Генерация

Дом,	который	построил	Джек.
Дом,	который	построил	Джек.
Дом,	который	построил	Джек.

Число уникальных токенов среди k токенов

$$\frac{|\text{unique tokens}|}{10} = \frac{4}{10}$$

Чем выше, тем больше разнообразие

Статистические метрики

Что если модель просто **запомнила** обучающую выборку?

Обучающий корпус

Вот дом,
Который построил Джек.
А это пшеница,
Которая в тёмном чулане хранится
В доме,
Который построил Джек.

Генерация

Дом, который построил Джек.
Дом, который построил Джек.
Дом, который построил Джек.

Статистические метрики

Что если модель просто **запомнила** обучающую выборку?

Обучающий корпус

Вот дом,
Который построил Джек.
А это пшеница,
Которая в тёмном чулане хранится
В доме,
Который построил Джек.

Генерация

Дом, который построил Джек.
Дом, который построил Джек.
Дом, который построил Джек.

Доля совпадения сгенерированных n-грамм с
n-граммами обучающего корпуса

$$\frac{|\{\text{train 2-grams}\} \cap \{\text{gen 2-grams}\}|}{|\{\text{gen 2-grams}\}|} = 1$$

Чем меньше, тем меньше запоминание

Статистические метрики

Модель 1

разнообразие: 0.5
запоминание: 0.5

Модель 2

разнообразие: 0.3
запоминание: 0.4

Как сравнивать?

Необходимо ввести единую метрику

Перплексия

- Посчитаем вероятности текстов из тестовой выборки.
- Чем лучше модель, тем больше должна быть вероятность.

Обучающий корпус

Вот дом,
Который построил Джек.
А это пшеница,
Которая в тёмном чулане хранится
В доме,
Который построил Джек.

Тестовый корпус

А это весёлая птица-синица,
Которая часто ворует пшеницу,
Которая в тёмном чулане хранится
В доме,
Который построил Джек.

$p(\text{“А это весёлая птица-синица, ...”}) = ?$

Перплексия

$$PPL(x) = p(x_1, \dots, x_m)^{-\frac{1}{m}}$$

- Чем меньше, тем лучше.
- $\frac{1}{m}$ нужно для нормализации по длине текста.

Перплексия была придумана еще в 1977 году, но она до сих пор популярна

Перплексия сводится к кросс-энтропии

$$PPL(x) = p(x_1, \dots, x_m)^{-\frac{1}{m}}$$

$$CE(x) = -\frac{1}{m} \sum_{i=1}^m \log p(x_i | x_{i-1}, \dots, x_1)$$

$$\begin{aligned} \exp(CE(x)) &= \exp\left(\sum_{i=1}^m \log p(x_i | x_{i-1}, \dots, x_1)\right)^{-\frac{1}{m}} = \left(\prod_{i=1}^m p(x_i | x_{i-1}, \dots, x_1)\right)^{-\frac{1}{m}} = \\ &= p(x_1, \dots, x_m)^{-\frac{1}{m}} = PPL(x) \end{aligned}$$

Методы семплирования токенов

Методы семплирования токенов

Модель выдает распределение вероятностей токенов



Как выбрать токен из этого распределения?

Жадное семплирование

Greedy Sampling

Можно всегда выбирать токен с максимальной вероятностью

$$y_t = \operatorname{argmax}_{y \in V} p(y | y_{<t})$$

Плюсы:

- Максимизируем вероятность текста

Минусы:

- Теряем разнообразие текста

Жадное семплирование

Greedy Sampling

Можно всегда выбирать токен с максимальной вероятностью

$$y_t = \operatorname{argmax}_{y \in V} p(y | y_{<t})$$

Плюсы:

- Максимизируем вероятность текста

Минусы:

- Теряем разнообразие текста
 - Плохо, если генерация **безусловная**
 - Не страшно, если **условная** (seq2seq)
- 

Beam Search

Лучевой поиск

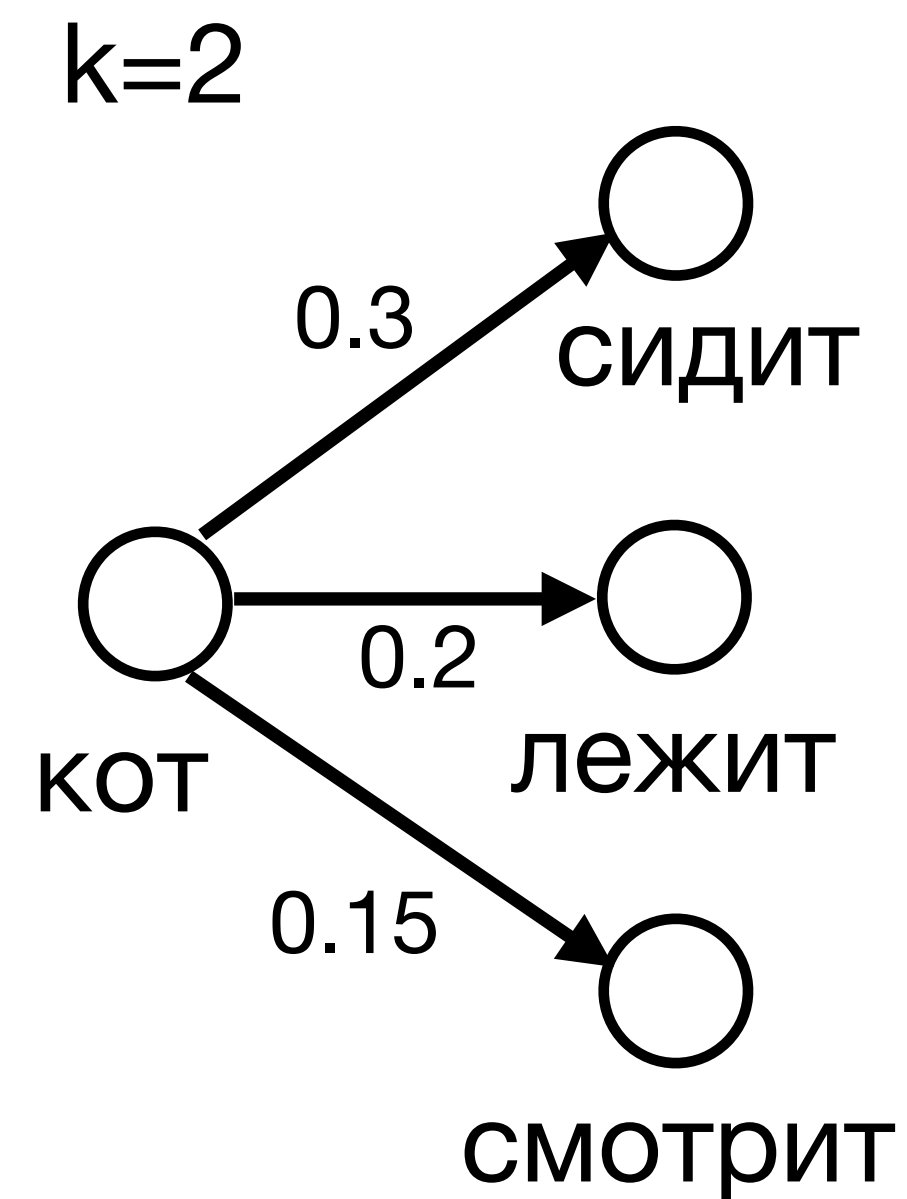
- Попробуем максимизировать вероятность текста еще больше
- При жадном семплировании мы **не учитываем** влияние токена на следующие за ним
- Будем строить предсказания на несколько токенов вперед, а затем выбирать токен с наибольшей совокупной вероятностью

Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

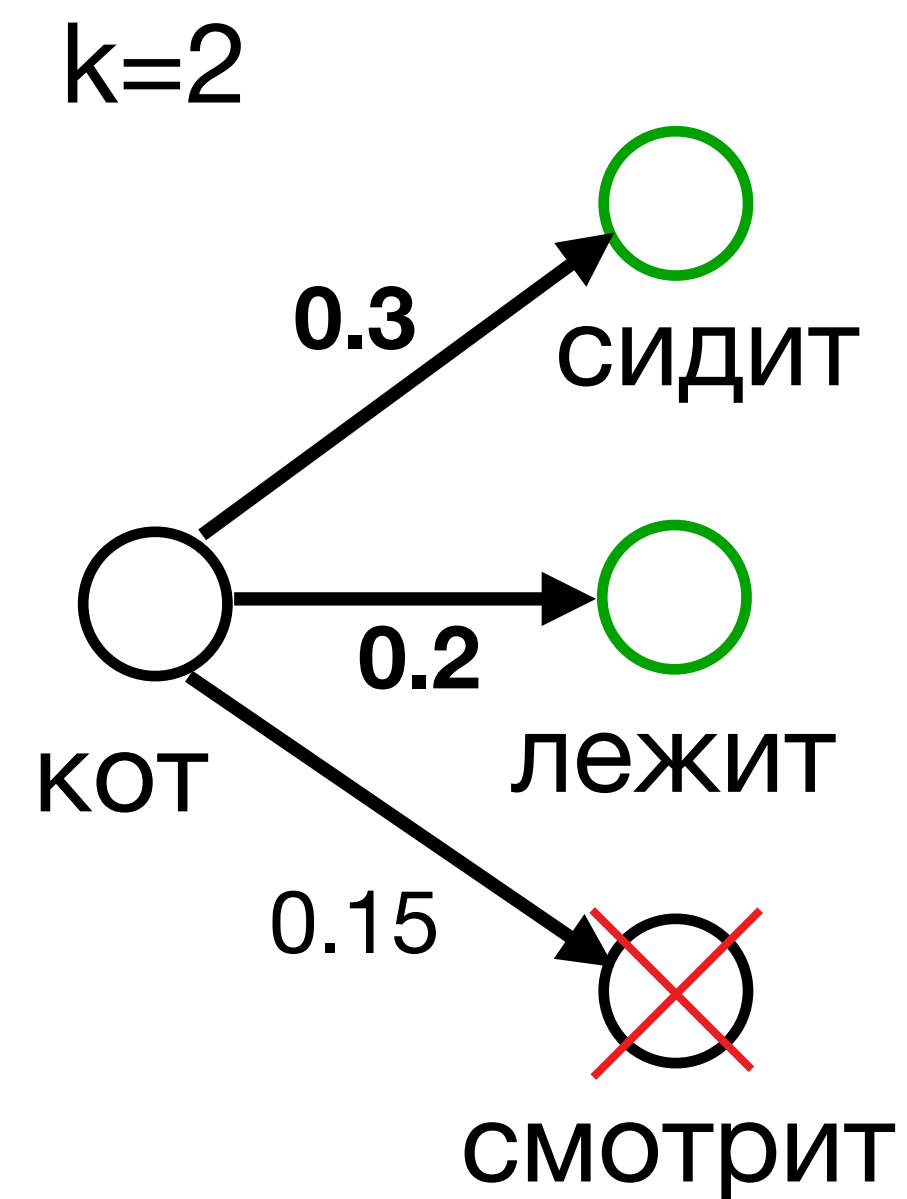


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

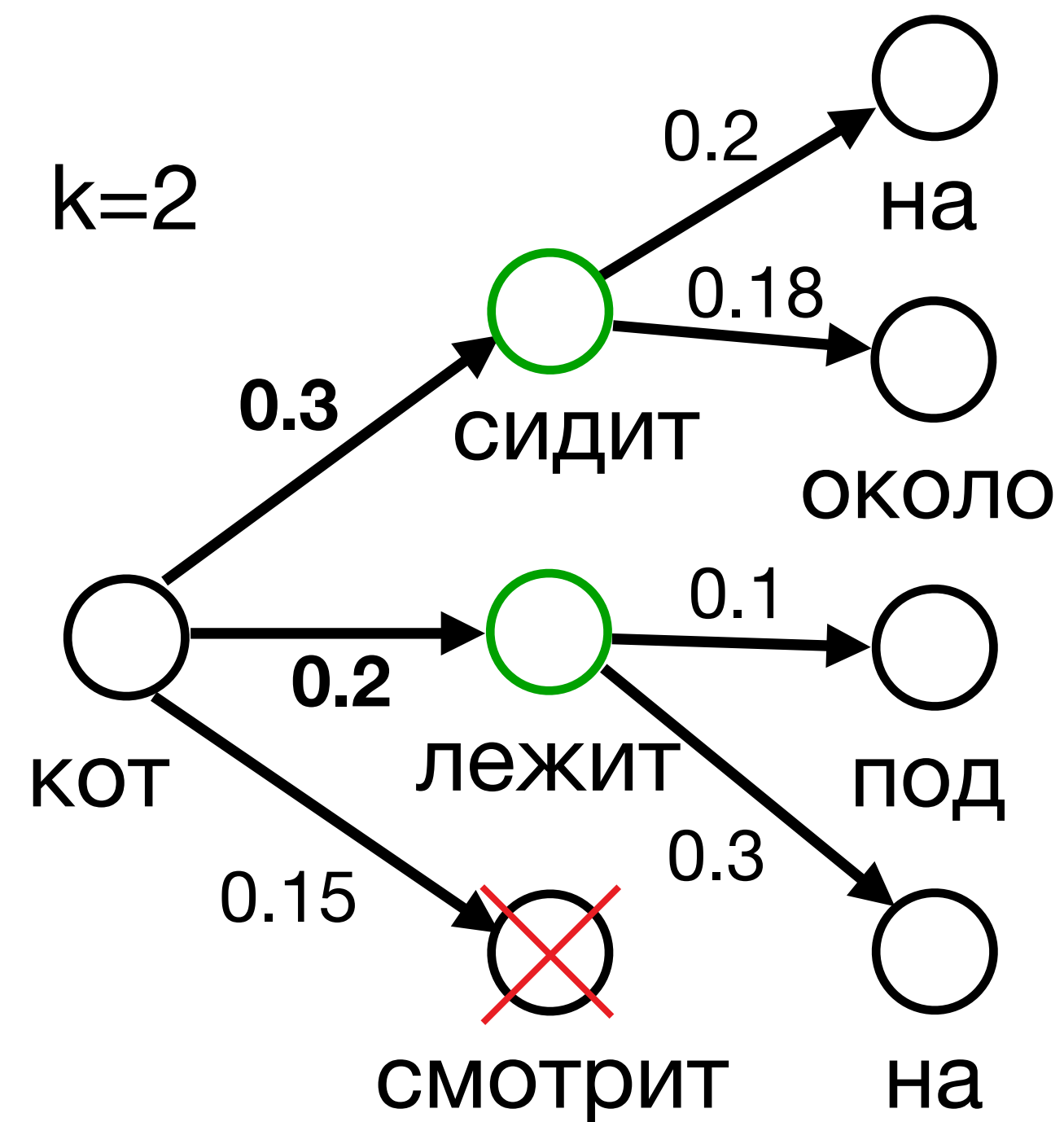


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

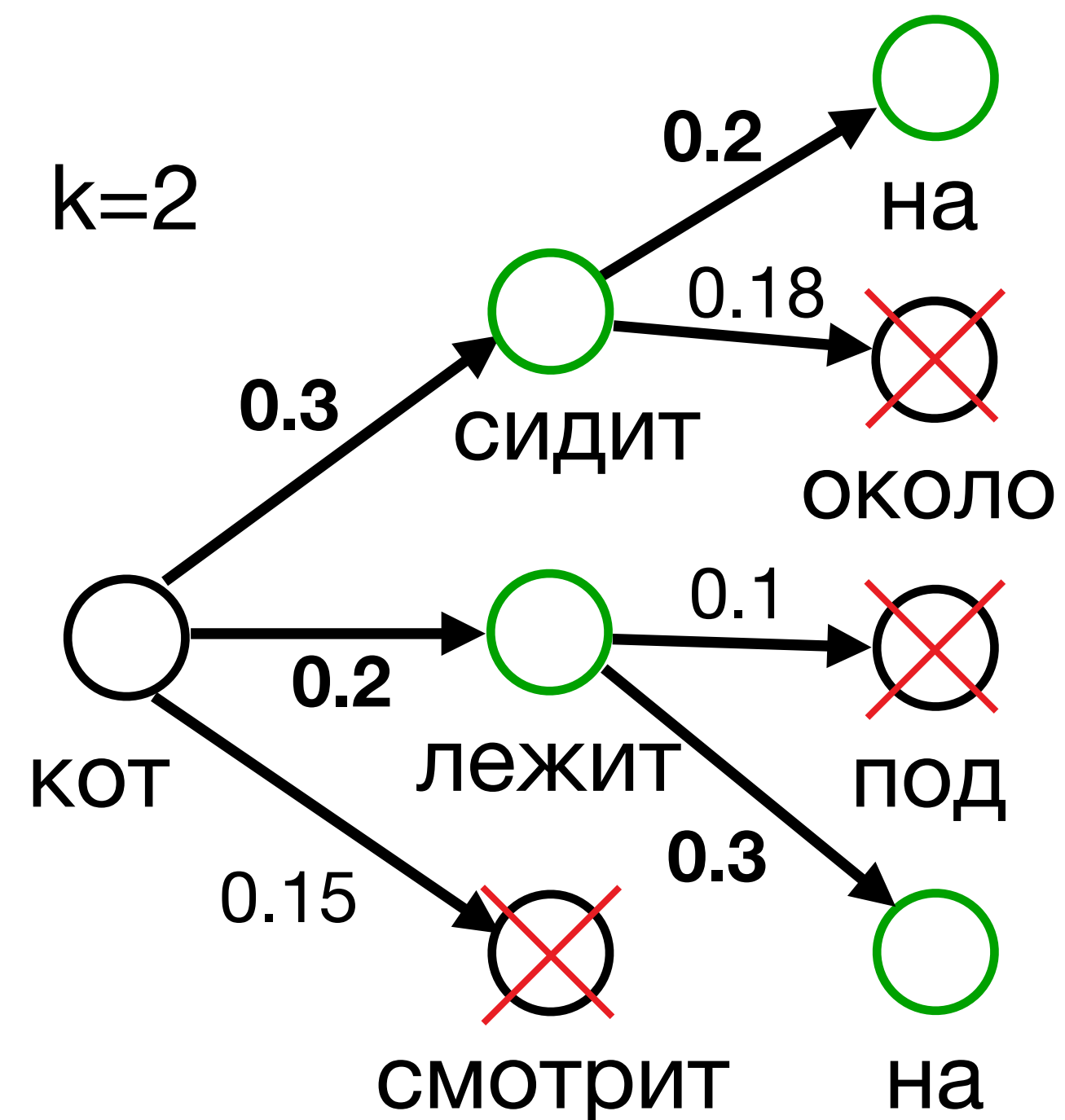


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

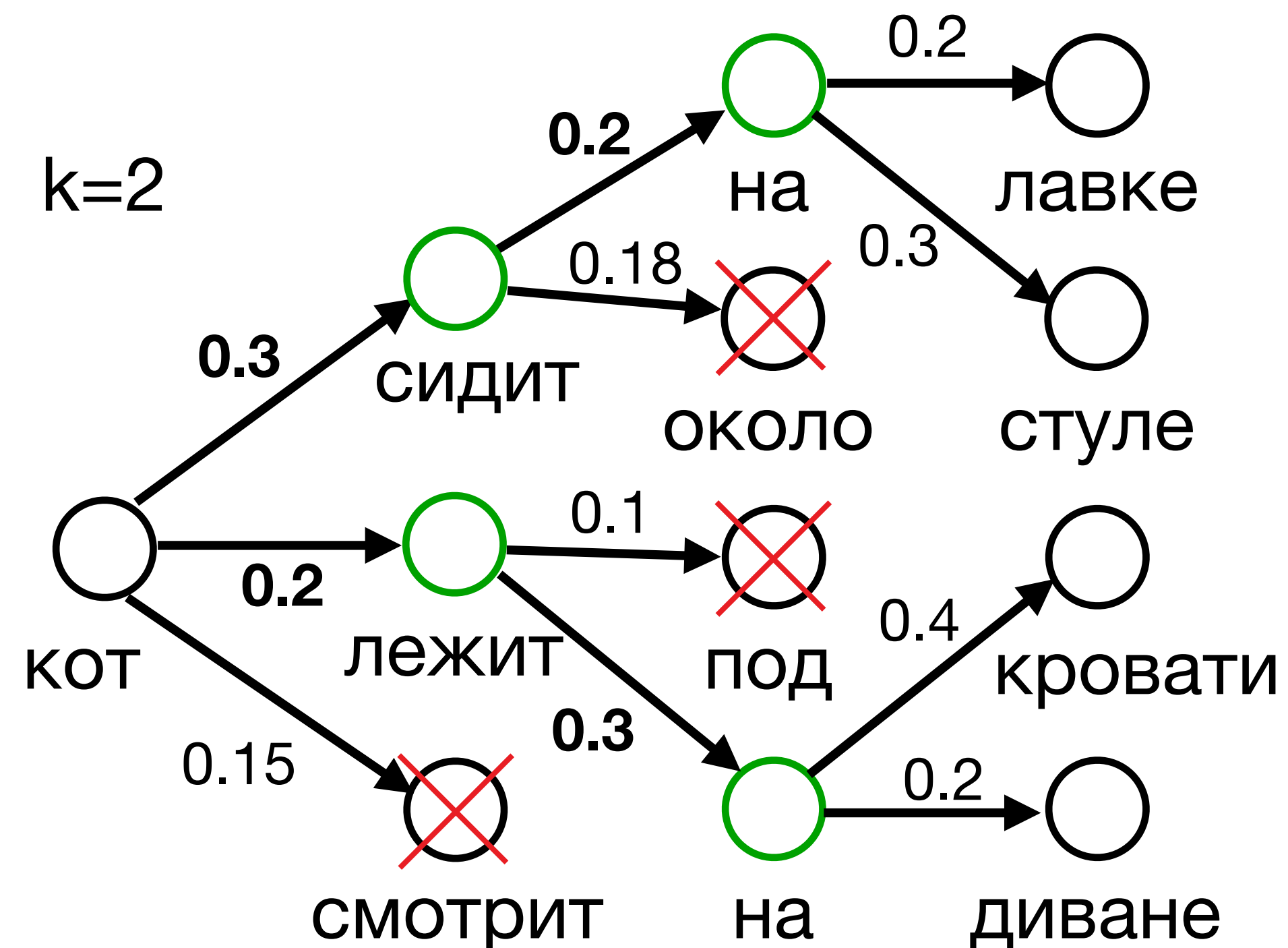


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

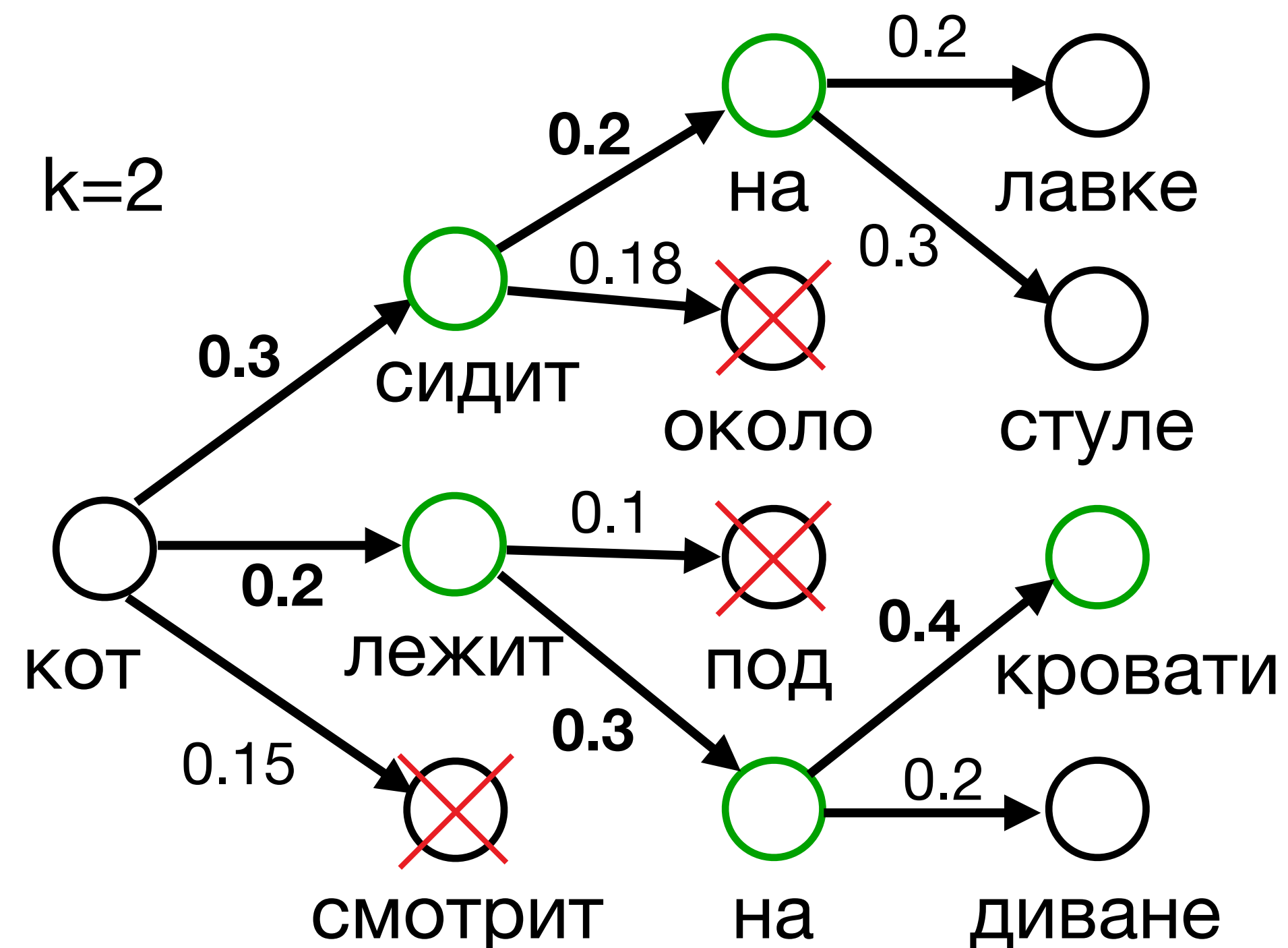


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных



Траектория с "лежит" имеет максимальную вероятность

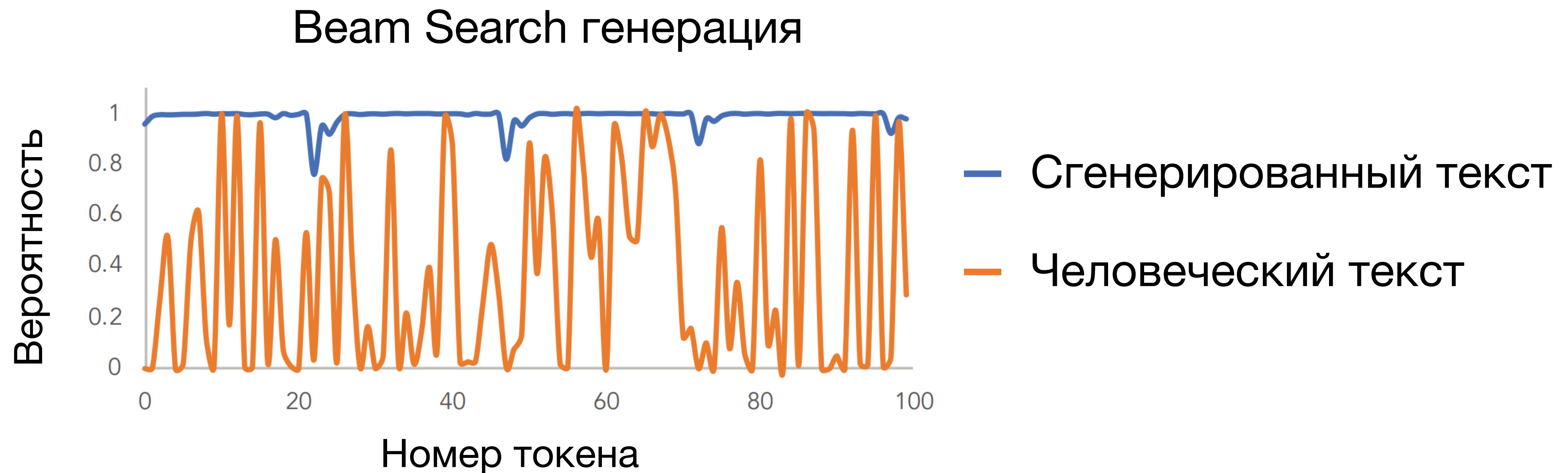
=> выбираем "лежит"

Beam Search vs Жадное семплирование

- Beam Search работает лучше для всех seq2seq задач
- Beam Search работает гораздо медленнее (зависит от k)
- Популярные значения k – 3 или 5

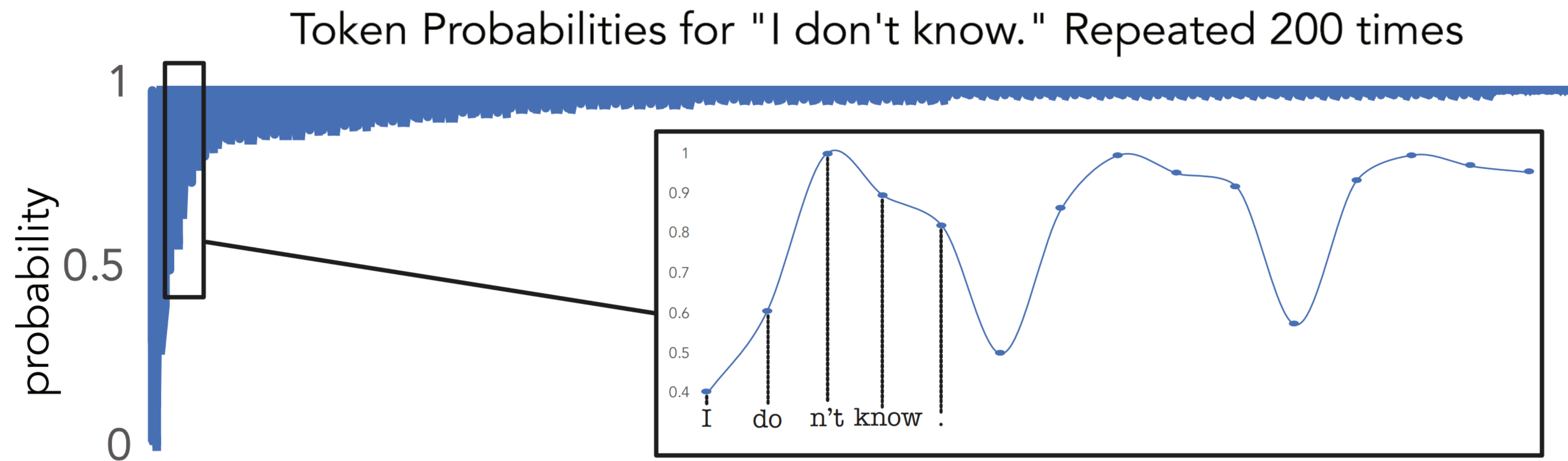
Безусловная генерация

Оба метода уменьшают разнообразие безусловной генерации!



Безусловная генерация

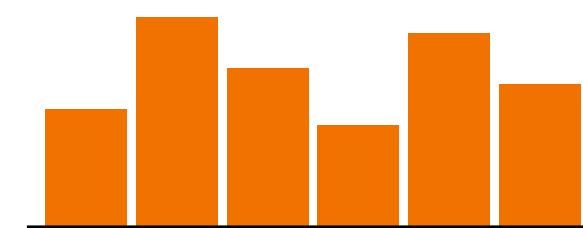
Оба метода поощряют повторения!



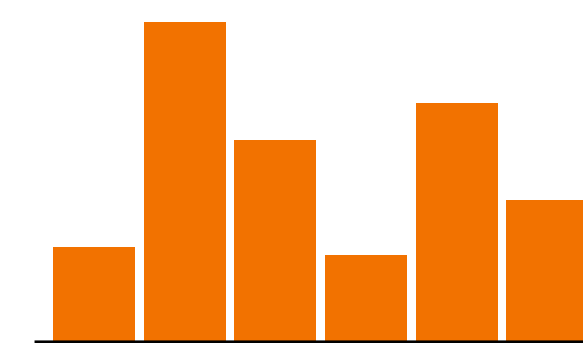
Семплирование с температурой

- При обычном семплировании с вероятностями вероятностная масса случайных токенов слишком велика
- Сделаем распределение более вырожденным, добавив температуру $\tau \in [0,1]$

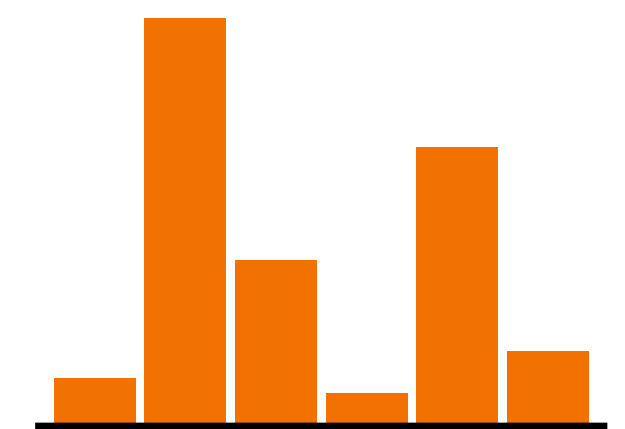
$$p_{\tau}(y | y_{<t}) = \frac{\exp \frac{p(y | y_{<t})}{\tau}}{\sum_{w \in V} \exp \frac{p(w | y_{<t})}{\tau}}$$



$p_2(y_t | y_{<t})$



$p(y_t | y_{<t})$

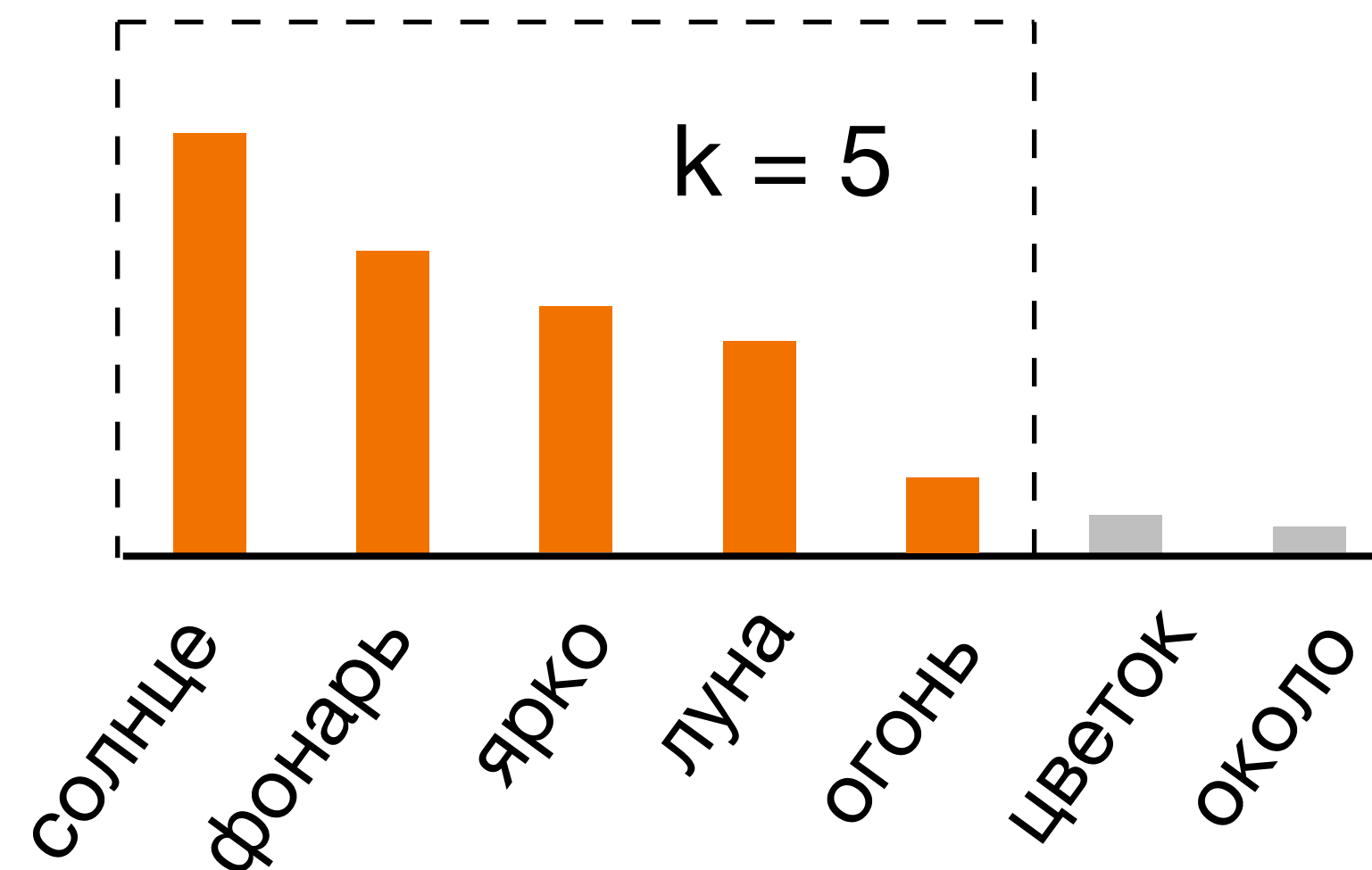


$p_{0.5}(y_t | y_{<t})$

Top-k семплирование

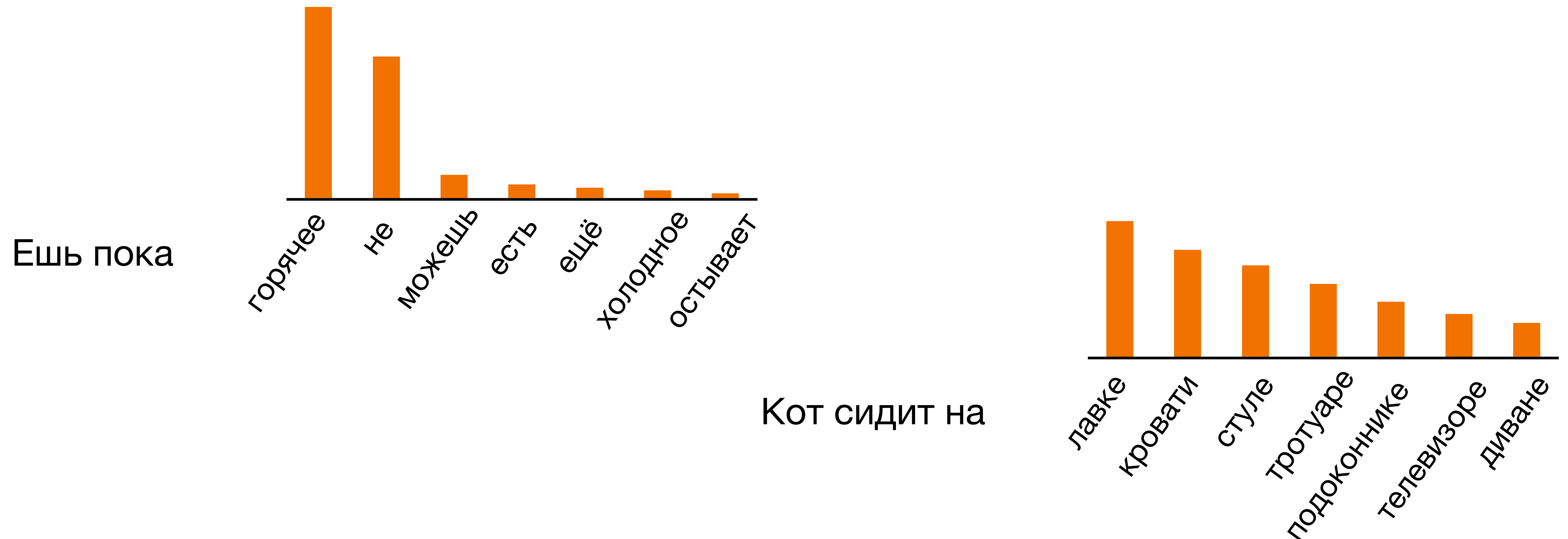
- Даже с температурой остается ненулевая генерация выдать случайный токен
- Оставим только k самых вероятных токенов и будем семплировать из них

На улице светит



Top-k семплирование

- Не во всех ситуациях самых вероятных токенов одинаковое число
- Из-за этого невозможно подобрать идеальное k



Тор-р семплирование

Nucleus sampling

- Будем выбирать из минимального числа токенов, суммарная вероятность которых больше p .

$$\sum_{w \in V^{(p)}} p(w | y_{<t}) \geq p$$

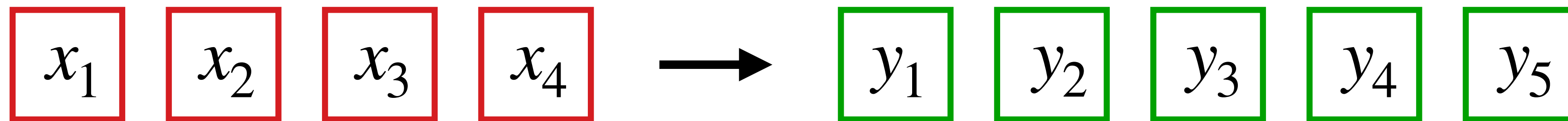
$$| V^{(p)} | \rightarrow \min$$

Популярное значение для p – 0.9 или 0.95

Seq2seq

Задача Seq2seq

Перевод входной последовательности в новую последовательность



Длины последовательностей могут **отличаться**

Примеры Seq2seq

1. Машинный перевод

Русский ↔ Английский

2. Суммаризация

Длинный текст → короткий текст

3. Изменение стиля текста

Грубый ↔ Вежливый

Формальный ↔ Неформальный

Формальная постановка задачи

Генерация текста:

$$p(x_1, \dots, x_m) = \prod_{i=1}^m p(x_i | x_{<i})$$

Формальная постановка задачи

Генерация текста:

$$p(x_1, \dots, x_m) = \prod_{i=1}^m p(x_i | x_{<i})$$

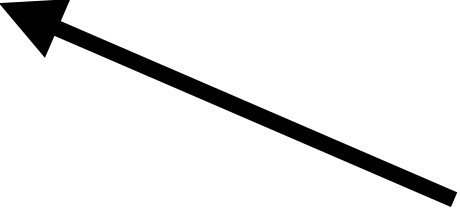
Seq2seq: добавляется условие

$$p(y_1, \dots, y_n | x_1, \dots, x_m) = \prod_{i=1}^n p(y_i | y_{<i}, x_1, \dots, x_m)$$

x – входная последовательность

y – новая (выходная) последовательность

учимся приближать

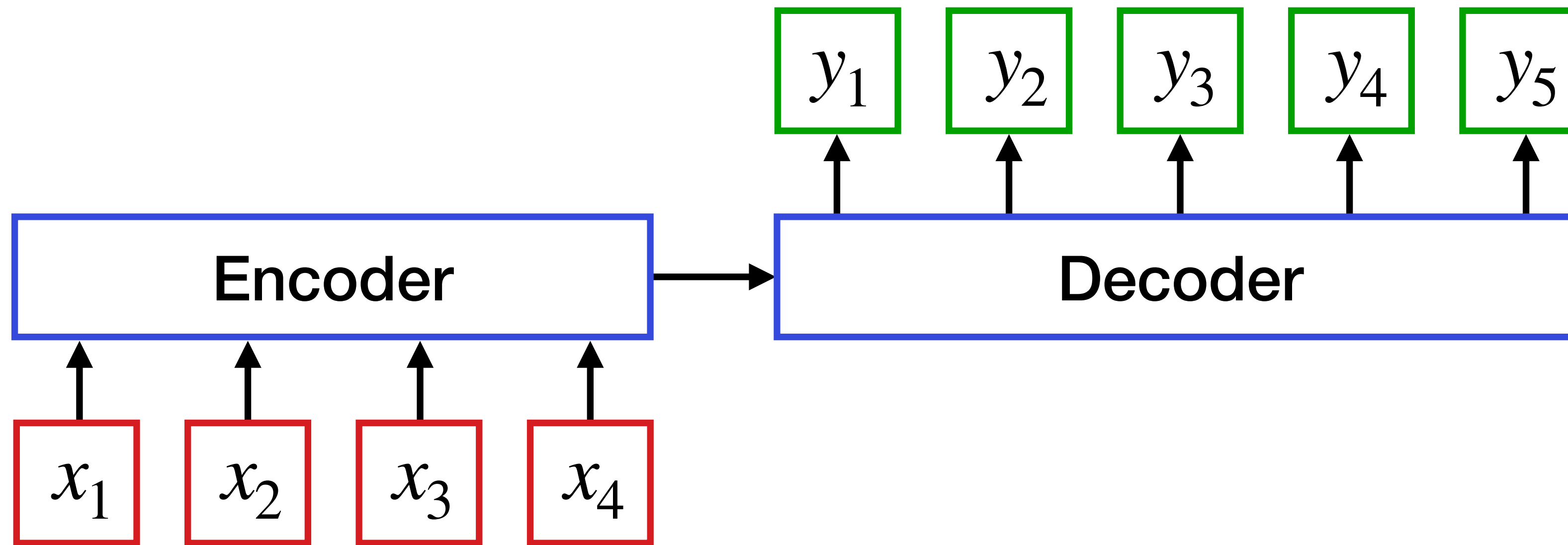


Семплирование

- В seq2seq не требуется генерировать разнообразный текст
- Нужно получить лучший текст по входному
- В seq2seq используется **beam search** или **жадное** семплирование
- В безусловной генерации – семплирование с **температурой** и **top-p**

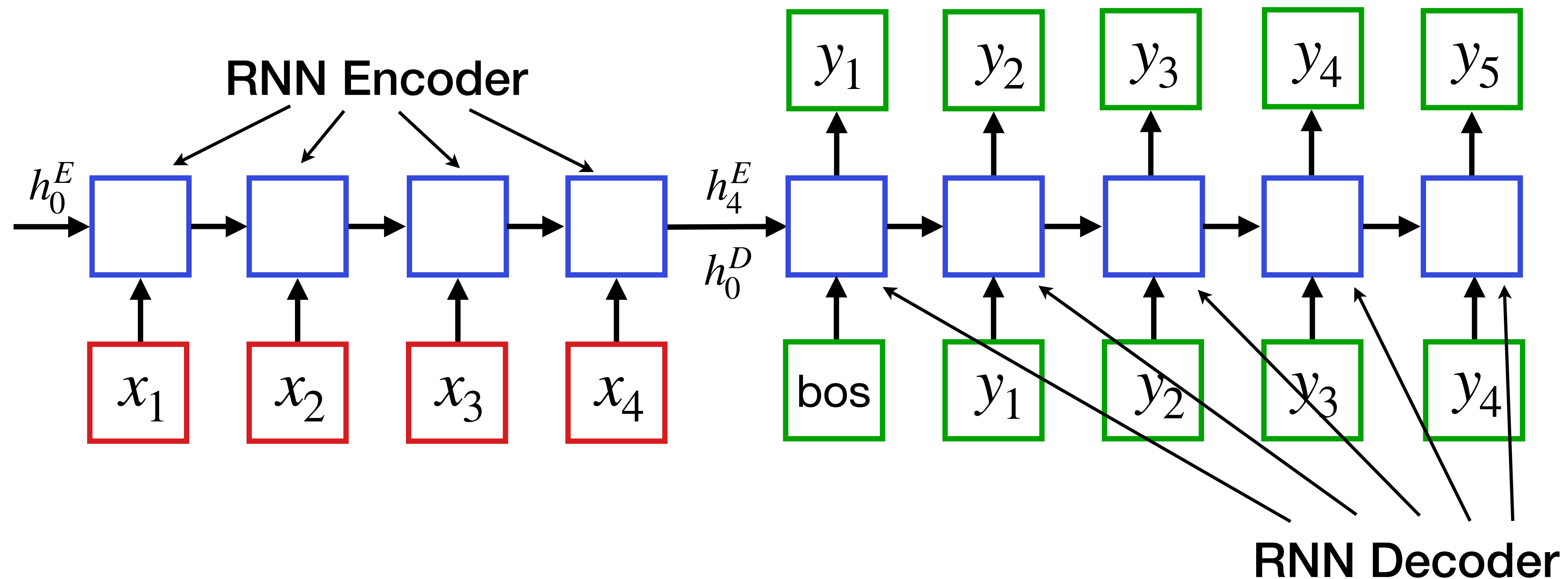
Стандартный подход

- Входная последовательность кодируется моделью **Encoder**
- **Decoder** принимает выход кодировщика и генерирует новую последовательность



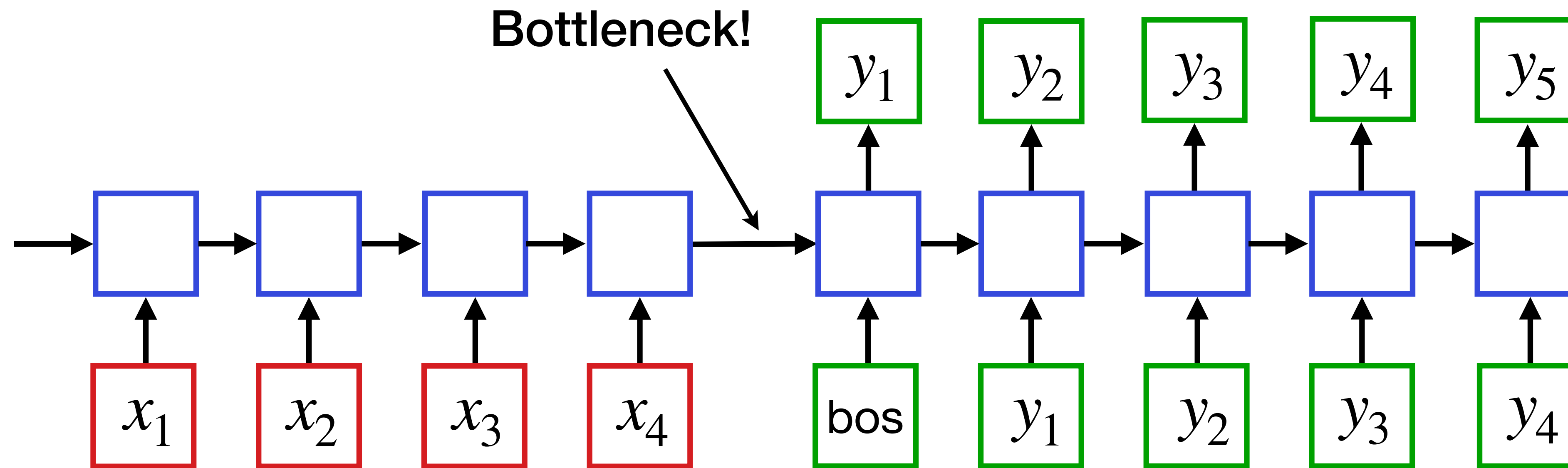
RNN подход

- Одной RNN моделью кодируем текст и получаем вектор состояния h^E
- Второй RNN моделью генерируем последовательность



RNN подход: проблема

- Входной текст может иметь произвольную длину
- Мы пытаемся уместить всю информацию о нем в один вектор h^E
- Получаем бутылочное горлышко (bottleneck)



Механизм внимания

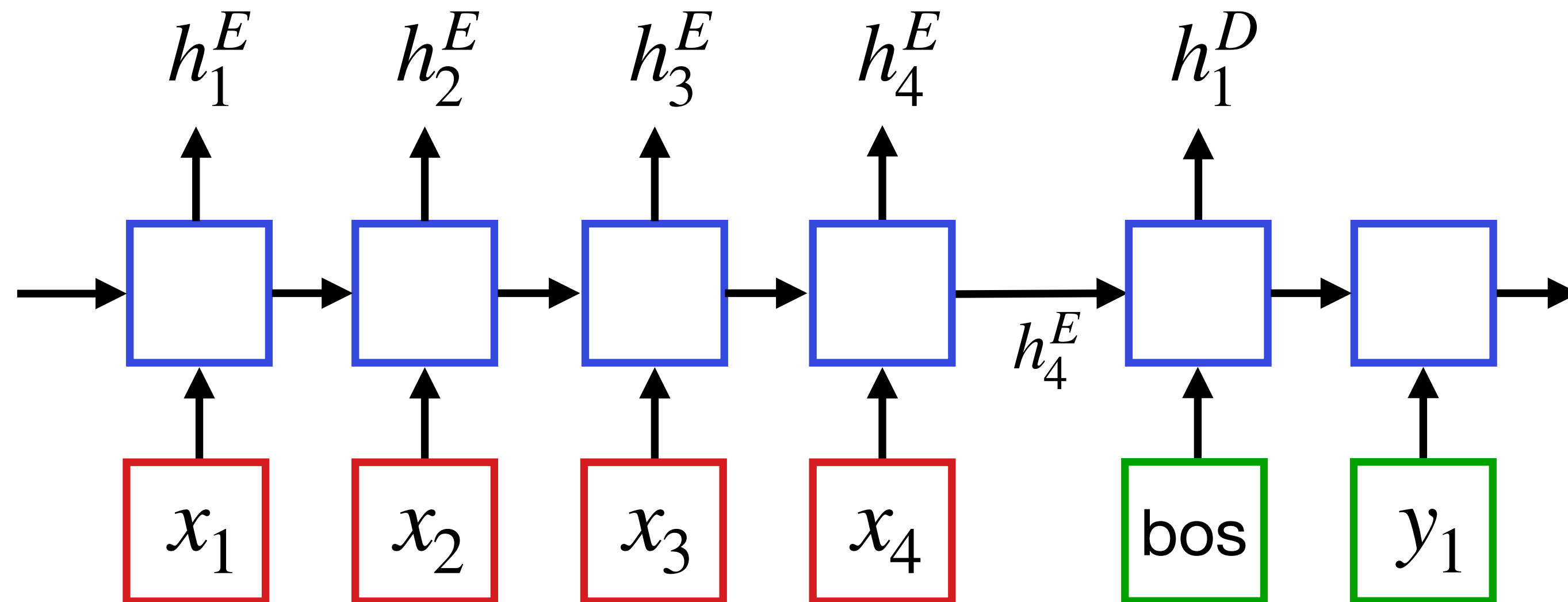
Идея:

Позволим декодеру "смотреть" на произвольные выходы кодировщика

Механизм внимания

Идея:

Позволим декодеру "смотреть" на произвольные выходы кодировщика

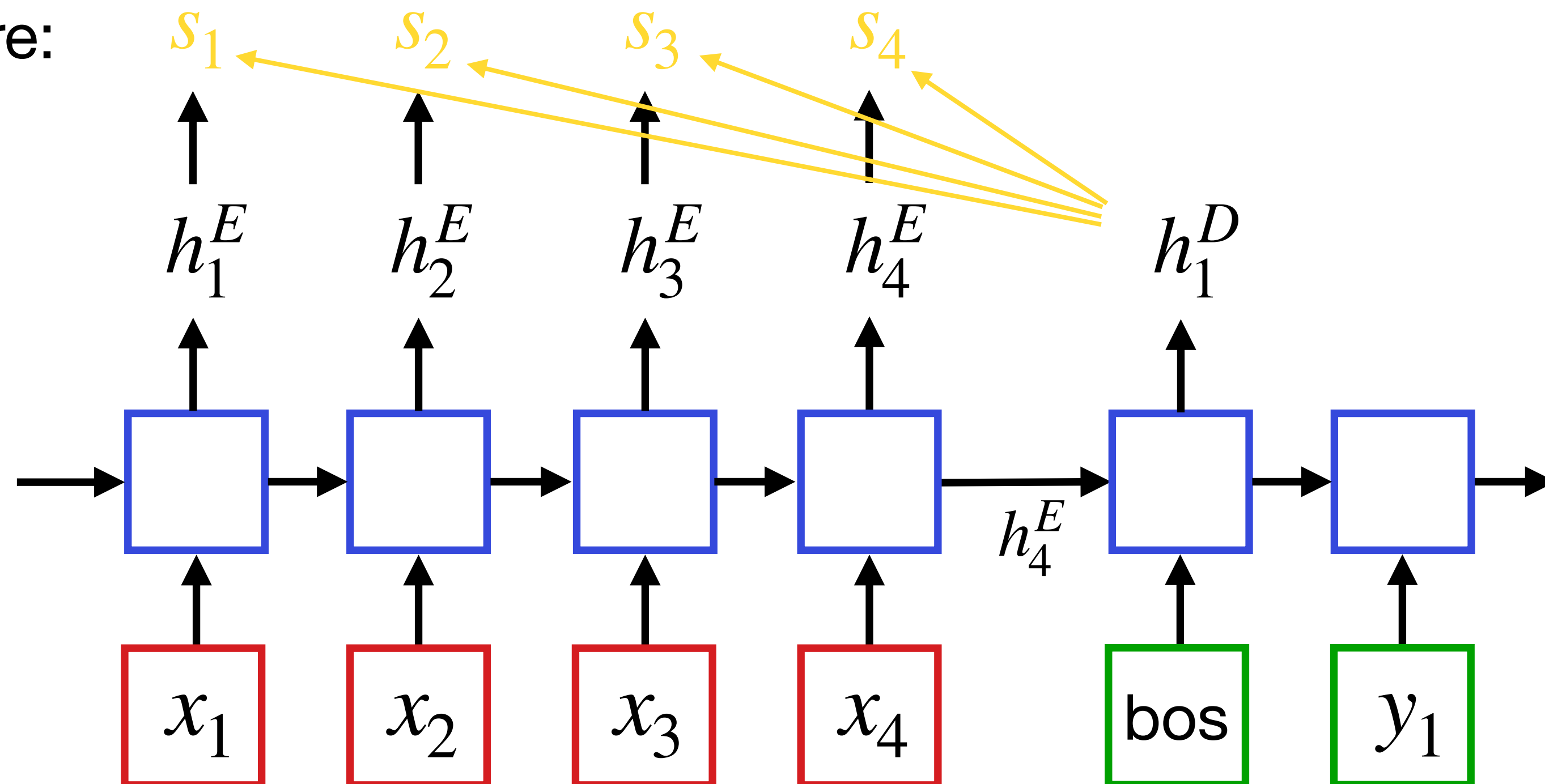


Механизм внимания

Идея:

Позволим декодеру "смотреть" на произвольные выходы кодировщика

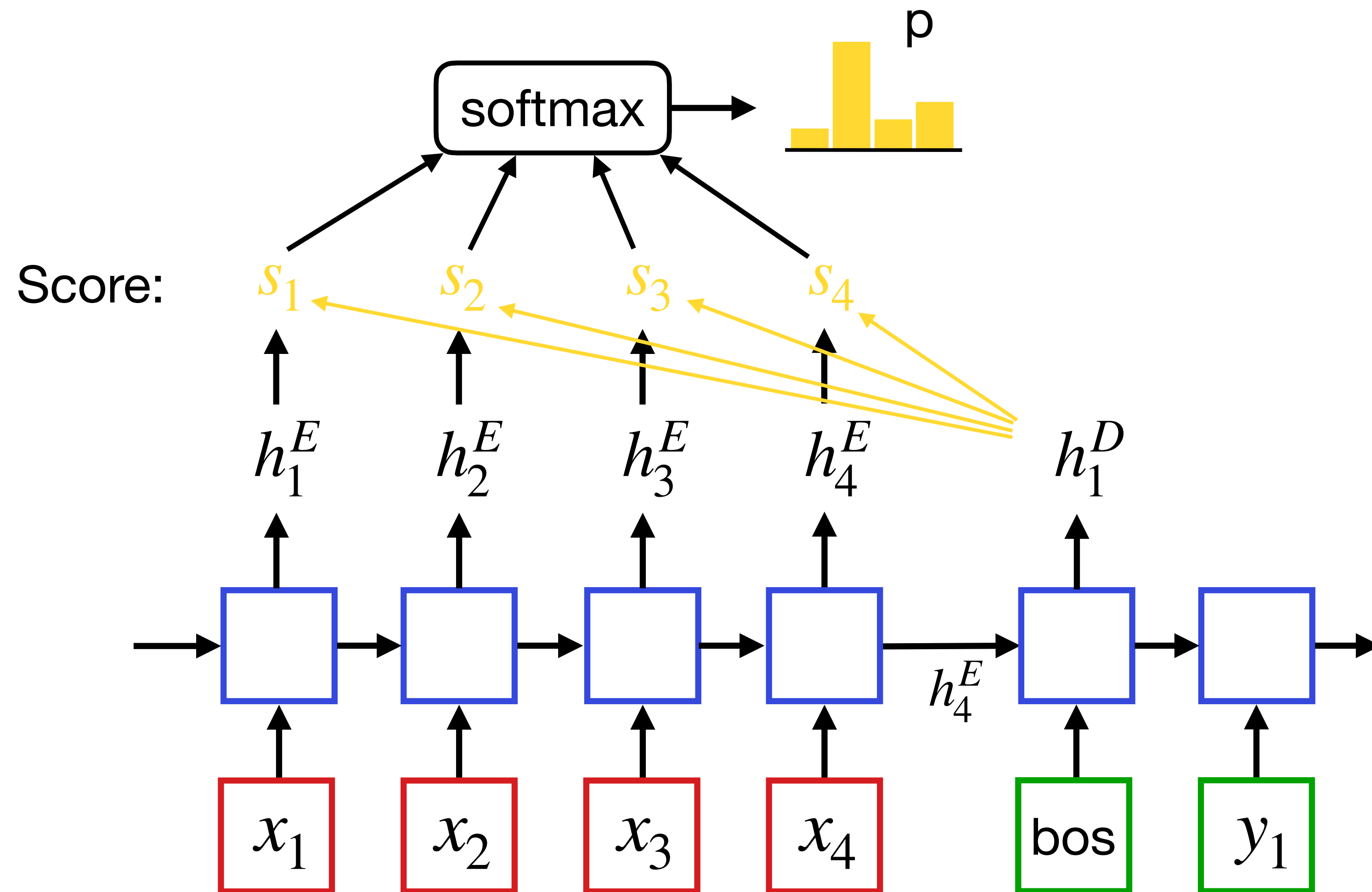
Score:



Механизм внимания

Идея:

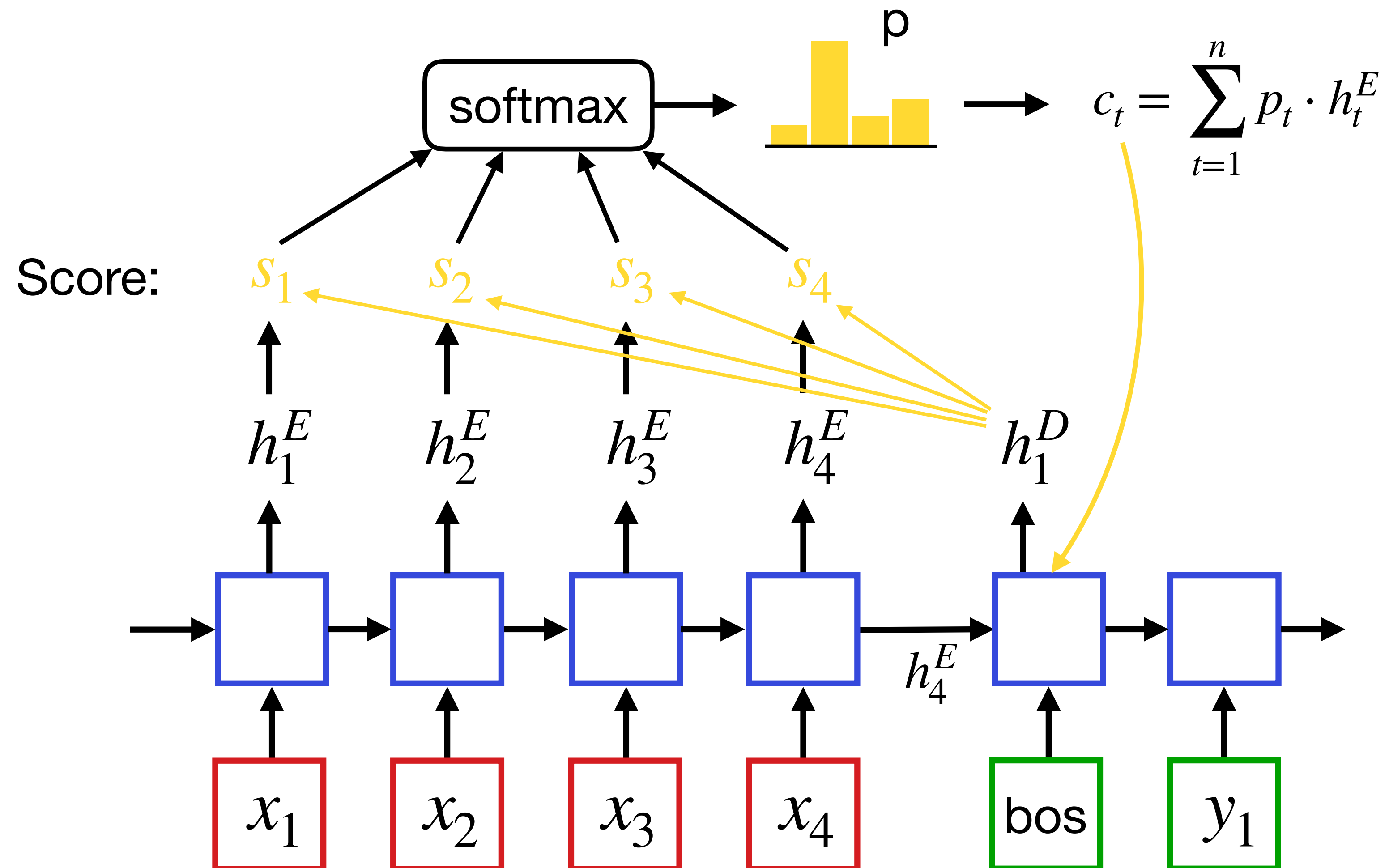
Позволим декодеру "смотреть" на произвольные выходы кодировщика



Механизм внимания

Идея:

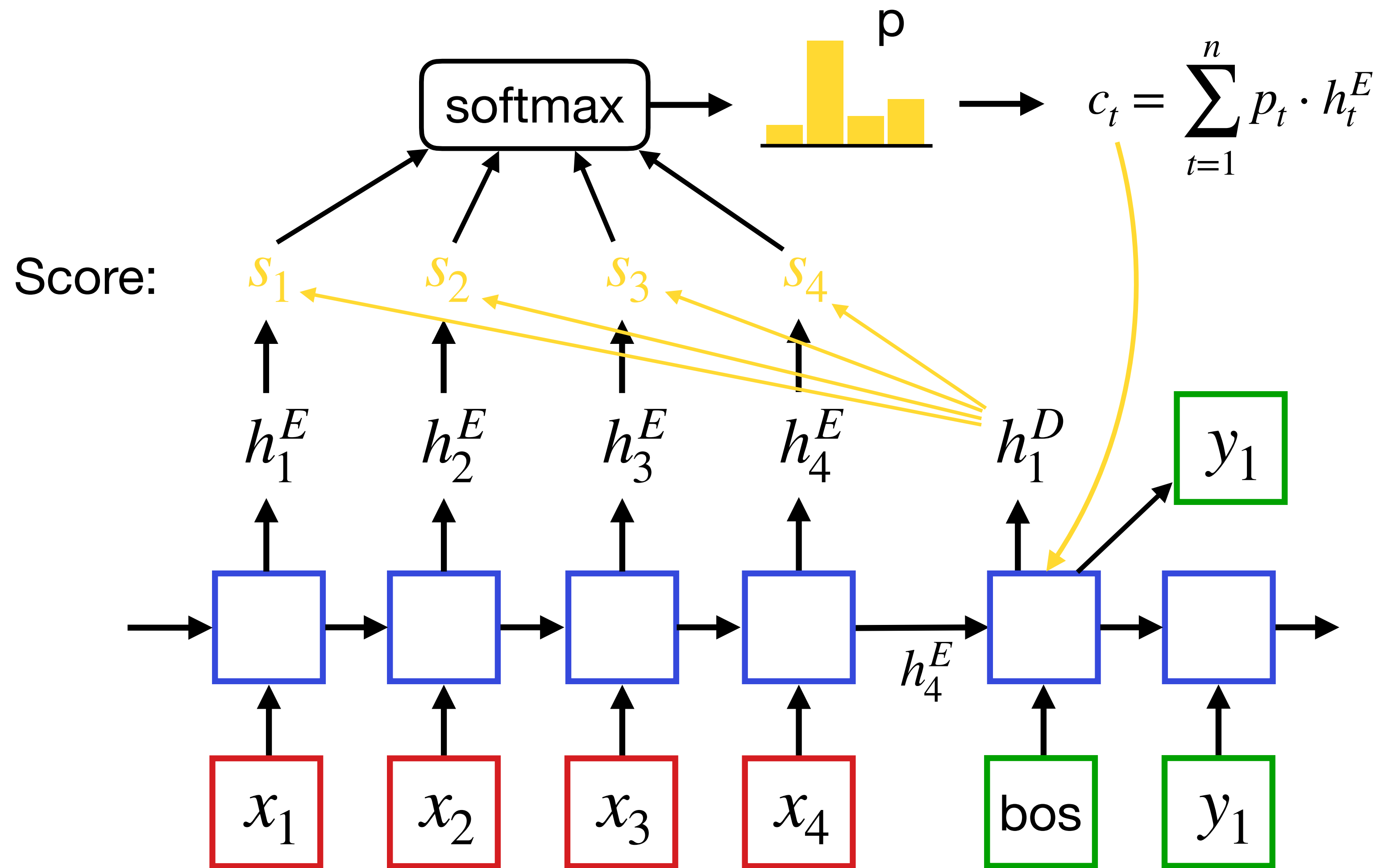
Позволим декодеру "смотреть" на произвольные выходы кодировщика



Механизм внимания

Идея:

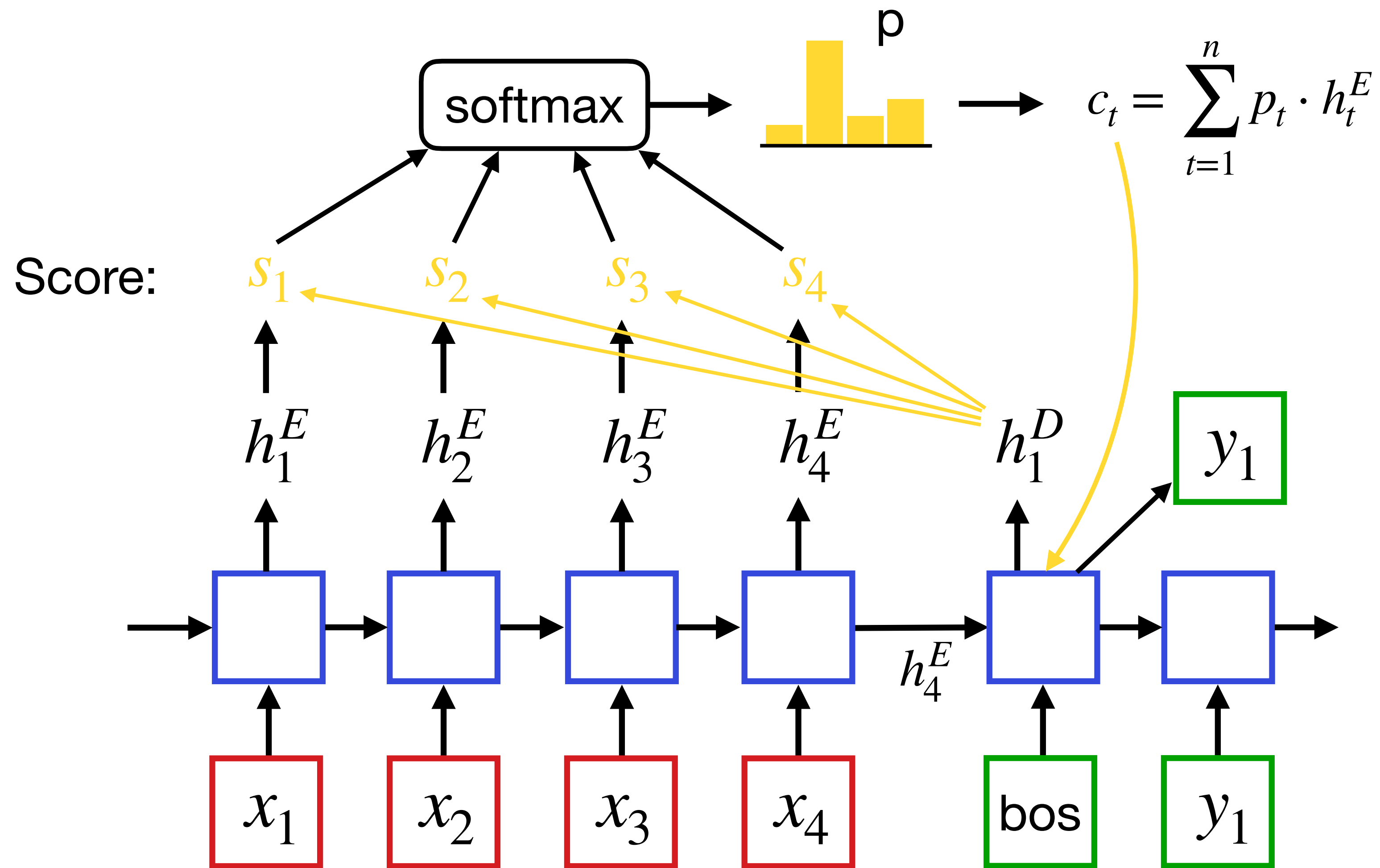
Позволим декодеру "смотреть" на произвольные выходы кодировщика



Механизм внимания

Идея:

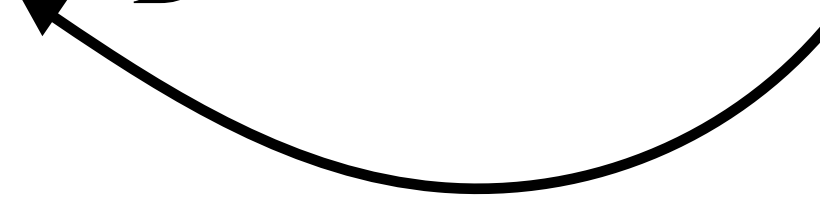
Позволим декодеру "смотреть" на произвольные выходы кодировщика



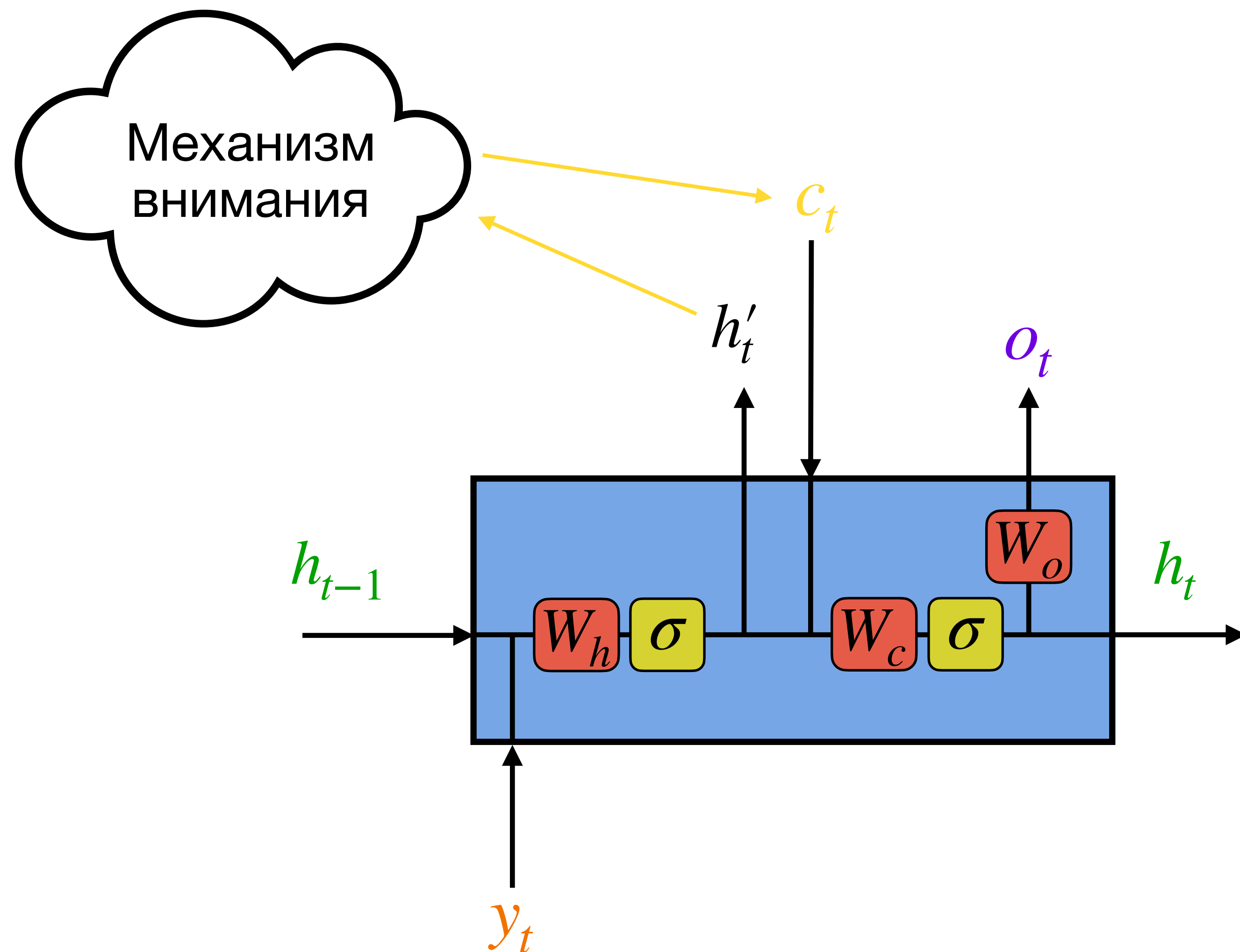
- Что такое Score?
- Как обрабатывается c_t ?

Методы подсчета Score

$$s(h_E, h_D) \in \mathbb{R}$$

1. Скалярное произведение: $s(h_E, h_D) = \langle h_E, h_D \rangle$
 2. Билинейная форма: $s(h_E, h_D) = h_E^T W h_D$ Обучаемая матрица
 3. Любая произвольная функция
- 

Блок декодера



$$h'_t = \tanh(W_h[y_t, h_{t-1}] + b_h)$$

$$h_t = \tanh(W_c[c_t, h'_t] + b_c)$$

$$o_t = W_o h_t + b_o$$

Можно комбинировать и по-другому:
[<https://arxiv.org/abs/1409.0473>, Bahdanau et al, 2014]
[<https://arxiv.org/abs/1508.04025>, Luong et al, 2015]

Метрики качества

Seq2seq

Метрики качества Seq2seq

- Для оценки качества Seq2seq модели логично сравнить сгенерированный текст с правильным
- Обычно есть несколько подходящих текстов
- Поэтому требовать точное совпадение глупо
- Чаще всего считают совпадение **n-грамм** слов
- Самая известная метрика – **BLEU**

BLEU: BiLingual Evaluation Understudy

Была придумана в **2001** и до сих пор остается самой популярной метрикой

Применяется для

- Машинного перевода
- Диалоговых систем
- Суммаризации текстов

BLEU: Как считать?

BLEU основана на **precision**

$\hat{y}$ – сгенерированный текст

y – правильный текст

Precision

$p_n(\hat{y}, y)$ = доля n-грамм слов в $\hat{y}$, которые присутствуют в y .

Если правильных текстов несколько, то проверяется присутствие в каждом тексте.

BLEU: Как считать?

BLEU основана на **precision**

$\hat{y}$ – сгенерированный текст

y – правильный текст

Precision

$p_n(\hat{y}, y)$ = доля n-грамм слов в $\hat{y}$, которые присутствуют в y .

Если правильных текстов несколько, то проверяется присутствие в каждом тексте.

Пример 1:

$\hat{y}$: рыжий кот прилег на лавку

$y^{(1)}$: кот лежит на лавке

$y^{(2)}$: на лавку легла кошка

$$p_1(\hat{y}, y) = \frac{3}{5}$$

BLEU: Как считать?

BLEU основана на **precision**

$\hat{y}$ – сгенерированный текст

y – правильный текст

Precision

$p_n(\hat{y}, y)$ = доля n -грамм слов в $\hat{y}$, которые присутствуют в y .

Если правильных текстов несколько, то проверяется присутствие в каждом тексте.

Пример 2:

$\hat{y}$: рыжий кот прилег на лавку

$y^{(1)}$: кот лежит на лавке

$y^{(2)}$: на лавку легла кошка

$$p_2(\hat{y}, y) = \frac{1}{4}$$

Precision для корпуса текстов

$\hat{Y} = (\hat{y}_1, \dots, \hat{y}_m)$ – корпус сгенерированных текстов

$Y = (y_1, \dots, y_m)$ – корпус правильных текстов

У каждого правильного текста может быть несколько вариантов.

$$y_i = (y_i^{(1)}, \dots, y_i^{(k)})$$

$$p_n(\hat{Y}, Y) = \frac{\sum_{i=1}^m |\text{пересечение } n\text{-грамм слов из } \hat{y}_i \text{ и } y_i|}{\sum_{i=1}^m |\text{n-граммы слов в } \hat{y}_i|}$$

Для максимизации **precision** разумно генерировать очень короткие тексты!

Штраф за краткость (Brevity penalty)

$$BP(\hat{Y}, Y) = \exp\left(-\left[\frac{\sum_{i=1}^m |y_i|}{\sum_{i=1}^m |\hat{y}_i|} - 1\right]_+\right)$$

длины правильных
текстов

длины сгенерированных
текстов

Если сгенерированные тексты суммарно

✓ Длиннее $\rightarrow BP(\hat{Y}, Y) = 1$

✗ Короче $\rightarrow BP(\hat{Y}, Y) = \exp\left(1 - \frac{\sum_{i=1}^m |y_i|}{\sum_{i=1}^m |\hat{y}_i|}\right) < 1$

BLEU: Итоговая метрика

$$\begin{aligned} BLEU(\hat{Y}, Y) &= BP(\hat{Y}, Y) \cdot \exp \left(\sum_{n=1}^N w_n \ln p_n(\hat{Y}, Y) \right) \\ &= BP(\hat{Y}, Y) \cdot \prod_{n=1}^N p_n(\hat{Y}, Y)^{w_n} \end{aligned}$$

- w_n – веса для разных n-грамм. Подбираются вручную.
- Обычно считают **1,2,3,4-граммы**.

$$BLEU(\hat{Y}, Y) \in [0,1]$$

Чем **больше**, тем **лучше**.

BLEU: Недостатки

- Не важен порядок слов и их смысл
- Плохо работает для морфологически богатых языков
- Требуется много эталонных текстов
- Много различных имплементаций. Отличия в
 - Токенизации
 - Методах сглаживания

Улучшения BLEU

ROUGE – F1-мера вместо precision

Использование:

- Суммаризация (почти все датасеты)
- Перефразирование

METEOR – F1-мера, учитывается порядок слов, их формы и синонимы

Использование:

- Машинный перевод

Другие метрики

- Есть масса других метрик, которые исправляют некоторые недостатки предыдущих
- Все они крайне редко используются

Метрики:

- eBLEU
- NIST
- HPPR
- LEPOR
- RIBES